FR

Bienvenue sur Sakata

Ce portail est un sanctuaire numérique dédié à la préservation et à la transmission de l'identité culturelle du peuple **Sakata**.

Notre objectif est d'unir nos racines ancestrales avec la splendeur numérique moderne, en offrant à la fois des archives vivantes, des sagesses intemporelles et un forum d'échange intergénérationnel.

Suivant →

> *Un éditeur d'articles c'est trompeusement simple à imaginer mais redoutable à implémenter. Le bon équilibre entre liberté éditoriale et structure se trouve dans le pattern « blocks » — popularisé par Notion, devenu standard de l'édition web moderne.*

DÉFINITION

Qu'est-ce que c'est ?

Accessible via `/admin/content/[slug]` (édition) ou `/admin/content/new` (création), l'éditeur permet aux admins/managers de composer des articles riches et multilingues.

- **4 types de blocs** : paragraphe, titre, image, citation.
- **6 langues** en parallèle (fr, en, skt, lin, swa, tsh) via tabs.
- **Drag-and-drop** pour réordonner les blocs.
- **Médiathèque** latérale pour insérer des images existantes en un clic.
- **Vidéo héro** optionnelle (voir doc dédiée).
- **Traduction auto** via Gemini — depuis FR vers les 5 autres langues.
- **Synthèse vocale** auto via Gemini Voice (option « Auto Voix »).

DÉCISION

Pourquoi le pattern blocks et pas Markdown brut ?

CHOIX

Modèle de données : `ContentBlock[]` sérialisé en JSONB Postgres, plutôt qu'un blob Markdown.

ASPECT	MARKDOWN BRUT	BLOCKS STRUCTURÉS	VERDICT
Stockage	TEXT simple	JSONB structuré	Égal
Édition WYSIWYG	Difficile (parser bidir)	Naturel	Blocks
Validation IA	Devine la structure	Structure explicite	Blocks
Migration de format	Tout casse	Schema versioning facile	Blocks
Reorder de paragraphes	Couper/coller	Drag-drop natif	Blocks
Mode legacy	Lecture immédiate	Rendu via composant	Markdown

Un mode **« Markdown brut »** est conservé en bascule (bouton `toggleMode`) pour les utilisateurs qui préfèrent — utile pour les articles très longs où le block editor devient lourd.

MODÈLE DE DONNÉES

Type ContentBlock

CODE

TYPESCRIPT

```typescript
export type ContentBlock = {
  id: string;
  type: "text" | "heading" | "image" | "quote";
  body?: string;       // text, heading, quote
  url?: string;        // image only
  caption?: string;    // image, quote
  alignment?: "left" | "full" | "right" | "sidebar";  // image only
};

// Stocké en DB :
articles.content = {
  fr: ContentBlock[],
  en: ContentBlock[],
  // ...
}
```

IMPLÉMENTATION

Drag-and-drop : Framer Motion Reorder

Plutôt que `react-dnd` (lourd, complexe), on utilise `<Reorder.Group>` de Framer Motion. Une seule prop `values + onReorder` suffit.

CODE TSX

```tsx
<Reorder.Group axis="y" values={blocks} onReorder={onChange}>
  {blocks.map(block => (
    <Reorder.Item key={block.id} value={block}>
      {/* Contenu éditable */}
    </Reorder.Item>
  ))}
</Reorder.Group>
```

BONUS

Le réordonnancement est animé fluidement (60fps) sans config supplémentaire. Sur mobile, tactile-friendly out-of-the-box.

IA

Traduction automatique

Pourquoi Gemini et pas Claude ou GPT ?

CHOIX

Gemini 1.5 Pro est le routage par défaut pour les traductions multilangues, en particulier vers les langues bantu.

Tests empiriques sur 20 articles types :

LANGUE CIBLE	GEMINI 1.5 PRO	CLAUDE 3.5 SONNET	GPT-4
Anglais	9/10	10/10	9/10
Lingala	8/10	6/10	5/10
Swahili	9/10	7/10	7/10
Kisakata	7/10	4/10 (refuse souvent)	5/10
Tshiluba	8/10	5/10	6/10

Gemini a clairement été entraîné sur plus de données africaines. Pour les langues bantu, c'est lui qui passe.

Préservation du glossaire

Certains termes ne doivent pas être traduits : *Ngongo*, *Mboka*, *Likonde*. Le prompt Gemini contient un glossaire explicite avec instructions de conservation.

IA

Synthèse vocale (Auto Voix)

Bouton « Auto Voix » dans le header de l'éditeur → appel à `/api/admin/ai/voice` qui utilise Gemini Voice Synthesis. Le fichier MP3 résultant est stocké et lié à l'article via `has_narrator = true`.

- Voix masculine grave par défaut, ton narratif (style « conteur »).
- Génération en arrière-plan, ~30s pour un article de 1000 mots.
- L'admin doit toujours sauvegarder l'article après génération pour persister `has_narrator`.

RETOUR D'EXPÉRIENCE

Difficultés rencontrées

Sérialisation JSONB et casts en TypeScript

Postgres renvoie le JSONB tel quel, mais TypeScript ne peut pas inférer la forme. Le pattern `article.content[lang] as ContentBlock[]` est utilisé partout — c'est la seule façon propre de typer.

Mode bascule text↔blocks et perte de données

Le toggle `text ↔ blocks` tente de convertir, mais la conversion blocks→text est lossy (un image block devient juste un `![](url)`). Avertissement utilisateur ajouté avant la bascule.

React Hook Form + Zod imbriqués

Validation des champs simples (titre, slug) via Zod. Validation des blocs imbriqués déléguée à des helpers manuels — Zod sur des structures profondes devient vite illisible.

À SAVOIR

Limitations connues

- Pas de versioning des articles (l'historique est perdu à chaque save).
- Pas d'édition collaborative (deux admins en édition = dernier qui save écrase).
- Pas d'embed YouTube/SoundCloud comme bloc dédié.

« Moto oyo ayebi te kotombola miso na likolo, akoki te koyeba esika oyo mbula eutaka. » — Proverbe Sakata

SAKATA

© 2026 — Brume de la Rivière. Tous droits réservés.

CULTURE ECOLE LANGUE HISTOIRE FORUM